

## Propostas Machine Learning & Chatbot no Projeto Tech

### Diretrizes Gerais e Arquitetura Elegível

- **Multidisciplinaridade:** A aplicação principal (interface de usuário, persistência relacional e regras de negócio base) é desenvolvida nas disciplinas de Web Design, Front-End e Back-End.
- **Obrigatoriedade de ML & Chatbot:** O módulo de inteligência deve ser desenvolvido em Python e isolado como um microsserviço desacoplado exposto via API REST (FastAPI ou Flask).
- **IA Generativa:** O uso de APIs externas (como OpenAI, Gemini ou Claude) não substitui a construção do pipeline de Machine Learning e processamento supervisionado/não-supervisionado em Python. O modelo próprio é o core mandatório da nota.

### Propostas Técnicas:

#### EIXO 1: CHATBOTS E PROCESSAMENTO DE LINGUAGEM NATURAL (NLP)

##### PROPOSTA 1: Assistente Virtual Transacional com NLU

**Conceito:** Assistente conversacional que interpreta intenções complexas do usuário e extrai entidades (parâmetros) para disparar ações de escrita no banco de dados da aplicação principal.

- **Tópicos da Ementa:** NLP, Regex, Lematização (spaCy), Classificação de Intenções (Scikit-Learn/Rasa NLU), Extração de Entidades (NER/Slots).
- **Arquitetura de Integração:**
  1. *Microsserviço Python:* Endpoint /nlu/parse recebe o texto, limpa, extrai a intenção e popula a estrutura de JSON de entidades.

2. *Back-End Web*: Consome a intenção traduzida e grava a transação na base relacional do sistema via webhook.
  3. *Front-End*: Widget interativo de chat com suporte a *Quick Replies* e botões dinâmicos.
- **Entregáveis Obrigatórios:**
    - Dataset próprio com intenções e 50 amostras por classe.
    - Pipeline de ML em Python com validação cruzada (*k*-fold) e Matriz de Confusão exportada.
    - Arquivo .pkl ou .joblib do modelo NLU carregado em memória no startup da API FastAPI.
    - Coleção do Postman com testes dos endpoints de inferência.

## **PROPOSTA 2: Chatbot c/Engine de busca semântica e resposta contextual (RAG)**

- **Conceito:** Bot especialista para resposta de dúvidas operacionais da plataforma com base em documentação técnica ou FAQs, garantindo precisão através de busca por similaridade vetorial.
- **Tópicos da Ementa:** Tokenização avançada, *Word/Sentence Embeddings* (GloVe/Sentence-Transformers), Similaridade de Cosseno, Matrizes de Distância Numérica.
- **Arquitetura de Integração:**
  1. *Microserviço Python*: Script de indexação que pré-processa os documentos do sistema e gera os vetores numéricos. Endpoint */faq/search* recebe a dúvida do usuário, vetoriza a mensagem e retorna o trecho com maior similaridade semântica.
  2. *Back-End Web*: Painel administrativo onde o gestor insere novos artigos; o microserviço refaz a indexação vetorial.

3. *Front-End*: Interface de busca conversacional com exibição do grau de confiança (similaridade em %) e citação da fonte.

- **Entregáveis Obrigatórios:**

- Base de conhecimento indexada contendo ao menos 20 documentos/perguntas técnicas.
- Script funcional de geração e persistência da matriz de embeddings.
- Endpoint REST operando com resposta < 200 ms para cálculo de similaridade semântica.
- Relatório de avaliação de performance do algoritmo frente a termos sinônimos.

### **PROPOSTA 3: Motor Conversacional com Triagem e Análise de Sentimento**

- **Conceito:** Canal de atendimento onde cada mensagem do cliente é analisada em tempo real para detectar seu tom emocional, redirecionando automaticamente chamados críticos no sistema.
- **Tópicos da Ementa:** TF-IDF / Bag-of-Words, Regressão Logística / Naive Bayes, Engenharia de Features de Texto, Métricas de Avaliação ( $F_1$ -score).
- **Arquitetura de Integração:**
  1. *Microserviço Python*: API REST com o modelo treinado para estimar o tom da mensagem (positivo, neutro, crítico) junto com a intenção.
  2. *Back-End Web*: Ao registrar um chamado com tom crítico, a API do Back-End eleva o *SLA* e o nível de prioridade no banco de dados.
  3. *Front-End*: Dashboard em tempo real exibindo os chamados organizados por nível de urgência e mapa de calor de sentimento dos usuários.
- **Entregáveis Obrigatórios:**
  - Dataset rotulado de análise de sentimento no domínio do projeto.

- Documentação da pipeline de pré-processamento (Regex, Stemming/Lemmatization, TF-IDF).
- Relatório de métricas de avaliação: Acurácia, Precisão, Recall e  $F_1$ -score.
- Endpoint em Python entregando o *payload* de diagnóstico com score numérico de sentimento.

## **EIXO 2: MODELOS PREDITIVOS DE MACHINE LEARNING**

### **PROPOSTA 4: Microserviço Preditivo de Métricas Contínuas (Regressão Supervisionada)**

- **Conceito:** Motor estatístico para estimar valores numéricos contínuos fundamentais para o negócio (ex.: estimativa de custo, precificação dinâmica, previsão de tempo de conclusão de serviço, consumo de recursos).
- **Tópicos da Ementa:** Regressão Linear / Múltipla, Normalização/Padronização (StandardScaler), Tratamento de Outliers, Métricas de Erro Contínuo (MAE, MSE, RMSE,  $R^2$ ).
- **Arquitetura de Integração:**
  1. *Microserviço Python:* Trata variáveis categóricas (One-Hot Encoding) e numéricas, treina o regressor e expõe o endpoint /predict/value.
  2. *Back-End Web:* Ao preencher um formulário na aplicação, o servidor consulta o microserviço Python para preencher automaticamente o valor sugerido.
  3. *Front-End:* Exibição gráfica da projeção calculada pela IA com limites de margem de erro estimada.
- **Entregáveis Obrigatórios:**
  - Dataset histórico simulado ou real com no mínimo 500 registros e 5 variáveis explicativas.
  - Notebook contendo a Análise Exploratória de Dados (EDA), matriz de

correlação de Pearson e remoção de outliers.

- Comparativo documentado de ao menos dois algoritmos de regressão com tabela de RMSE e  $R^2$ .
- Endpoint REST serializado pronto para receber *payloads* via POST.

## **PROPOSTA 5: Motor de Classificação e Decisão Automática com Limiares de Confiança**

- **Conceito:** Classificador supervisionado que recebe parâmetros do usuário ou transação para tomar decisões automáticas no fluxo de trabalho (ex.: risco de churn, pontuação de crédito/perfil, detecção de anomalias, priorização de tarefas).
- **Tópicos da Ementa:** KNN, Decision Trees / Random Forest, Divisão Treino/Teste, Matriz de Confusão, Curva ROC/AUC, Limiares de Decisão (Threshold Tuning).
- **Arquitetura de Integração:**
  1. *Microserviço Python:* Recebe o objeto via JSON, executa a padronização das features, aplica a classificação e retorna a classe prevista + a probabilidade calculada.
  2. *Back-End Web:* Caso a probabilidade seja menor que um determinado limite (ex.: < 70%), a aplicação exige aprovação manual do administrador.
  3. *Front-End:* Tags de status dinâmicas na interface e tela de auditoria das decisões automatizadas.
- **Entregáveis Obrigatórios:**
  - Modelo treinado, validado e exportado (.joblib).
  - Relatório técnico com Matriz de Confusão gráfica e análise do custo de Falsos Positivos vs. Falsos Negativos.
  - Lógica de código que retorne o valor de probabilidade (predict\_proba) via API REST.
  - Teste unitário em Python simulando requisições ao microserviço.

## PROPOSTA 6: Engine de Recomendação e Agrupamento de Perfil (K-Means)

- **Conceito:** Motor de aprendizagem não-supervisionada que analisa o comportamento, características ou consumo dos usuários para agrupá-los em *clusters* e gerar recomendações personalizadas na interface.
- **Tópicos da Ementa:** Aprendizado Não-Supervisionado, Algoritmo K-Means, Redução de Dimensionalidade / Normalização, Método do Cotovelo (*Elbow Method*), *Silhouette Score*.
- **Arquitetura de Integração:**
  1. *Microserviço Python:* Processa a base de dados tabulares, determina o número ideal de grupos, aloca o usuário em um *cluster* específico e retorna as entidades mais próximas do centroide.
  2. *Back-End Web:* Consome a lista de IDs recomendados e busca os detalhes no banco de dados para renderização rápida.
  3. *Front-End:* Vitrine "Recomendados Para Você" ou "Perfil Semelhante" totalmente alimentada pelo resultado do modelo.
- **Entregáveis Obrigatórios:**
  - Notebook com a justificativa visual do número  $K$  de clusters escolhido (Gráfico do Cotovelo + *Silhouette Score*).
  - Objeto da esteira de transformação (Pipeline do Scikit-Learn com StandardScaler e KMeans) salvo em arquivo binário.
  - Endpoint REST `/recommend/{user_id}` operacional.
  - Dashboard ou seção na aplicação mostrando o comportamento da vitrine com base no perfil atribuído ao usuário.

#### 4. Instruções e Prazos de Entrega

Para a formalização da proposta escolhida, a equipe deve seguir estritamente os passos abaixo:

- O tema escolhido tem que integrar com o que será construído com outras disciplinas do semestre.
- Prazo limite do envio da definição: 20/09/2026 às 23:59.
- Canal de Envio: E-mail para [flavio.santarelli@pro.fecaf.com.br](mailto:flavio.santarelli@pro.fecaf.com.br) com o assunto [PROJETO TECH] - Nome do Grupo / Turma.
- **Conteúdo Obrigatório do E-mail:**
  1. **Integrantes:** Nome completo e RA de todos os alunos do grupo.
  2. **Tema Geral do Projeto Tech:** Breve resumo da aplicação desenvolvida para Web Design, Front-End e Back-End.
  3. **Opção Escolhida:** Qual das 6 propostas acima foi selecionada ou detalhamento da proposta autoral para validação.
  4. **Repositório GitHub:** Link do repositório oficial da equipe onde ficarão hospedados todos os artefatos do projeto (documentação, códigos, scripts)

**Prof. Flavio Santarelli**